🧠 Face Recognition Attendance System

Technical Sheet
Traitz Academy | Hort 7.0 | Machine Learning Track

📦 INSTALLATION COMMANDS

```
# Install all required libraries at once
pip install opencv-python
pip install face-recognition
pip install numpy
pip install pandas
pip install Pillow
pip install streamlit

# If face_recognition fails install cmake first
pip install cmake
pip install dlib
pip install face-recognition

# Generate your requirements file
pip freeze > requirements.txt
```

If you get errors on Windows:

```
pip install face-recognition-models
pip install --upgrade pip setuptools wheel
```

🔗 OFFICIAL DOCUMENTATION LINKS

Library	Documentation	Purpose
OpenCV	https://docs.opencv.org	Camera, image, face detection
face_recognition	https://github.com/ageitgey/face_recognition	Face encoding and matching
Pandas	https://pandas.pydata.org/docs	CSV and data handling
NumPy	https://numpy.org/doc	Array and number operations
Streamlit	https://docs.streamlit.io	Web interface

Tkinter	https://docs.python.org/3/library/tkinter.html	Desktop interface

---

PHASE 1 — BASIC FACE DETECTION

```python
import cv2

# Load the haar cascade face detector
face_cascade = cv2.CascadeClassifier(
    cv2.data.haarcascades + 'haarcascade_frontalface_default.xml'
)

# Start webcam
cap = cv2.VideoCapture(0)

while True:
    ret, frame = cap.read()
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    # Detect faces
    faces = face_cascade.detectMultiScale(gray, 1.1, 4)

    # Draw rectangle around each face
    for (x, y, w, h) in faces:
        cv2.rectangle(frame, (x, y), (x+w, y+h), (0, 255, 0), 2)
        cv2.putText(frame, 'Face Detected', (x, y-10),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 255, 0), 2)

    cv2.imshow('Face Detection', frame)

    # Press Q to quit
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

Help:
https://docs.opencv.org/4.x/db/d28/tutorial_cascade_classifier.html

---

📸 PHASE 2 — REGISTER FACES

```python
import face_recognition
import os
import pickle

# Path to your registered faces folder
REGISTERED_FACES_PATH = 'registered_faces/'

def register_faces():
    known_encodings = []
    known_names = []

    # Loop through every image in the folder
    for filename in os.listdir(REGISTERED_FACES_PATH):
        if filename.endswith('.jpg') or filename.endswith('.png'):

            # Get the name from the filename
            name = os.path.splitext(filename)[0]

            # Load image and generate encoding
            image_path = os.path.join(REGISTERED_FACES_PATH, filename)
            image = face_recognition.load_image_file(image_path)
            encodings = face_recognition.face_encodings(image)

            if encodings:
                known_encodings.append(encodings[0])
                known_names.append(name)
                print(f'Registered: {name}')
            else:
                print(f'No face found in {filename} — skip')

    # Save encodings to file so you don't reload every time
    with open('encodings.pkl', 'wb') as f:
        pickle.dump((known_encodings, known_names), f)

    print(f'Total registered: {len(known_names)} faces')
    return known_encodings, known_names

register_faces()
```

Help:
https://github.com/ageitgey/face_recognition#usage

🔍 PHASE 3 — REAL TIME RECOGNITION

```python
import cv2
import face_recognition
import pickle
import numpy as np

# Load saved encodings
with open('encodings.pkl', 'rb') as f:
    known_encodings, known_names = pickle.load(f)

# Start webcam
cap = cv2.VideoCapture(0)

while True:
    ret, frame = cap.read()

    # Resize frame for faster processing
    small_frame = cv2.resize(frame, (0, 0), fx=0.25, fy=0.25)
    rgb_small_frame = cv2.cvtColor(small_frame, cv2.COLOR_BGR2RGB)

    # Find all faces in current frame
    face_locations = face_recognition.face_locations(rgb_small_frame)
    face_encodings = face_recognition.face_encodings(
        rgb_small_frame, face_locations
    )

    for face_encoding, face_location in zip(face_encodings, face_locations):

        # Compare to known faces
        matches = face_recognition.compare_faces(
            known_encodings, face_encoding, tolerance=0.5
        )
        name = 'Unknown'

        # Find the best match
        face_distances = face_recognition.face_distance(
            known_encodings, face_encoding
        )
        best_match_index = np.argmin(face_distances)
```

```
        if matches[best_match_index]:
            name = known_names[best_match_index]

        # Scale back up face location
        top, right, bottom, left = face_location
        top *= 4; right *= 4; bottom *= 4; left *= 4

        # Draw box and name
        color = (0, 255, 0) if name != 'Unknown' else (0, 0, 255)
        cv2.rectangle(frame, (left, top), (right, bottom), color, 2)
        cv2.putText(frame, name, (left, top - 10),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.8, color, 2)

    cv2.imshow('Face Recognition', frame)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

Help:
https://github.com/ageitgey/face_recognition/blob/master/examples/facerec_from_webcam_faster.py

---

✅ PHASE 4 — MARK ATTENDANCE

```python
import pandas as pd
import os
from datetime import datetime

ATTENDANCE_FILE = f'attendance_records/attendance_{datetime.now().date()}.csv'

# Track who has been marked today
already_marked = set()

def mark_attendance(name):
    if name == 'Unknown':
        return

    if name not in already_marked:
        already_marked.add(name)

        now = datetime.now()
        date = now.strftime('%Y-%m-%d')
        time = now.strftime('%H:%M:%S')

        # Load existing file or create new one
        if os.path.exists(ATTENDANCE_FILE):
            df = pd.read_csv(ATTENDANCE_FILE)
        else:
            df = pd.DataFrame(columns=['Name', 'Date', 'Time', 'Status'])

        # Add new record
        new_record = {
            'Name': name,
            'Date': date,
            'Time': time,
            'Status': 'Present'
        }
        df = pd.concat(
            [df, pd.DataFrame([new_record])],
            ignore_index=True
        )

        # Save back to CSV
        df.to_csv(ATTENDANCE_FILE, index=False)
        print(f'Marked present: {name} at {time}')

# Call this inside your recognition loop
# mark_attendance(name)
```

Help:
https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.to_csv.html

---

🖥️ PHASE 5 — STREAMLIT INTERFACE

```python
import streamlit as st
import cv2
import face_recognition
import pickle
import numpy as np
import pandas as pd
from datetime import datetime
import os

st.title('Face Recognition Attendance System')
st.subheader('Traitz Academy | Hort 7.0')

# Load encodings
with open('encodings.pkl', 'rb') as f:
    known_encodings, known_names = pickle.load(f)

FRAME_WINDOW = st.image([])
attendance_display = st.empty()

already_marked = set()
cap = cv2.VideoCapture(0)

run = st.checkbox('Start Camera')
```

```
while run:
    ret, frame = cap.read()
    rgb_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)

    face_locations = face_recognition.face_locations(rgb_frame)
    face_encodings = face_recognition.face_encodings(rgb_frame, face_locations)

    for face_encoding, face_location in zip(face_encodings, face_locations):
        matches = face_recognition.compare_faces(known_encodings, face_encoding)
        name = 'Unknown'

        face_distances = face_recognition.face_distance(known_encodings, face_encoding
        if len(face_distances) > 0:
            best_match = np.argmin(face_distances)
            if matches[best_match]:
                name = known_names[best_match]
                mark_attendance(name)

        top, right, bottom, left = face_location
        cv2.rectangle(rgb_frame, (left, top), (right, bottom), (0, 255, 0), 2)
        cv2.putText(rgb_frame, name, (left, top - 10),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 255, 0), 2)

    FRAME_WINDOW.image(rgb_frame)

    # Show attendance table
    if os.path.exists(ATTENDANCE_FILE):
        df = pd.read_csv(ATTENDANCE_FILE)
        attendance_display.dataframe(df)

cap.release()
```

Help:
https://docs.streamlit.io/library/api-reference

🐛 COMMON ERRORS AND FIXES

Error: No module named face_recognition

```bash
pip install cmake dlib face-recognition
```

Reference: https://github.com/ageitgey/face_recognition#installation

---

Error: No module named cv2

```
pip install opencv-python
```

Error: Dlib not installing on Windows

Download the pre-built dlib wheel for your Python version here:
https://github.com/z-mahmud22/Dlib_Windows_Python3.x

Error: Camera not opening — index 0

```python
# Try index 1 instead
cap = cv2.VideoCapture(1)
```

Error: Face not being detected in image

Make sure your registered photo:

- Has good lighting
- Shows a clear frontal face
- Is not too small or blurry
- Is saved as .jpg or .png only

Error: Everyone is being labelled Unknown

```python
# Lower the tolerance value (default is 0.6)
# Lower = stricter matching
matches = face_recognition.compare_faces(
    known_encodings, face_encoding, tolerance=0.5
)
```

Error: System is too slow

```python
# Resize the frame before processing
small_frame = cv2.resize(frame, (0, 0), fx=0.25, fy=0.25)
# Process only every 2nd frame
frame_count = 0
if frame_count % 2 == 0:
    # run recognition here
frame_count += 1
```

📐 KEY CONCEPTS EXPLAINED SIMPLY

Face Encoding
A face encoding is a list of 128 numbers that uniquely describes a face. Think of it as a face's fingerprint. Two photos of the same person will produce very similar numbers. Two different people will produce very different numbers.

Face Distance
Face distance measures how different two face encodings are. A distance of 0.0 means identical. Anything below 0.6 is usually a match. The lower the number the more confident the match.

Tolerance
Tolerance is the threshold that decides if two faces match. Default is 0.6. Lower it to make matching stricter. Raise it to make matching more relaxed.

Haar Cascade
A pre-trained XML file that OpenCV uses to detect faces quickly. It looks for patterns of light and dark areas that are common in human faces.

RGB vs BGR
OpenCV reads images in BGR format. face_recognition needs RGB format. Always convert before processing:

```python
rgb_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
```

📚 LEARNING RESOURCES

Video Tutorials:

- Face Recognition Full Project:
  https://www.youtube.com/watch?v=sz25xxF_AVE
- OpenCV Face Detection Beginner:
  https://www.youtube.com/watch?v=WlGKa6jqxDo
- Attendance System Full Build:
  https://www.youtube.com/watch?v=tl2eEBFEHqM
- Streamlit Beginners Guide:
  https://www.youtube.com/watch?v=JwSS70SZdyM

Reading:

- face_recognition full documentation:
  https://face-recognition.readthedocs.io/en/latest/face_recognition.html
- OpenCV Python tutorials:
  https://docs.opencv.org/4.x/d6/d00/tutorial_py_root.html
- Pandas read and write CSV:
  https://pandas.pydata.org/docs/getting_started/intro_tutorials/02_read_write.html
- Real Python OpenCV guide:
  https://realpython.com/face-recognition-with-python

If You Get Stuck:

- Stack Overflow:
  https://stackoverflow.com/questions/tagged/face-recognition
- OpenCV Forum:
  https://forum.opencv.org

- face_recognition GitHub Issues: https://github.com/ageitgey/face_recognition/issues

---

✅ DAILY CHECKLIST

Day 1:

- All libraries installed with no errors
- requirements.txt generated
- Webcam opens and detects face with green box

Day 2:

- Minimum 5 faces registered with clear photos
- encodings.pkl file generated successfully
- All names printing correctly in terminal

Day 3:

- Live camera recognises all 5 registered faces
- Unknown faces labelled correctly
- Name displays above face box in real time

Day 4:

- Attendance CSV file being created automatically
- Each person logged only once per session
- Date and time recording correctly

Day 5:

- Interface running cleanly
- Camera feed visible in interface
- Attendance table updating live
- Export button working

Day 6:

- Tested in different lighting
- Tested with glasses and different angles
- README written and explains how to run project
- Presentation ready

💡 PRO TIPS

- Always use good lighting when taking registration photos
- Take 2 to 3 photos of each person from slightly different angles for better accuracy
- Test your system with people wearing glasses and without
- Always convert BGR to RGB before passing to face_recognition
- Process smaller frames to keep the camera running smoothly
- Save your encodings to a pickle file so you do not reload every time you run the system
- Commit your code to GitHub every day so you never lose progress

---